

NATIONAL UNIVERSITY OF HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF COMPUTER ENGINEERING

CIRCUIT DESIGN WITH HDL

CHAPTER 3: STRUCTURAL MODELING

Ho Ngoc Diem

Course outline

- Chapter 1: Introduction
- Chapter 2: Verilog Language Concepts
- Chapter 3: Structural modeling
- Chapter 4: Behavioral modeling
- Chapter 5: State machines
- Chapter 6: Tasks and Functions
- Chapter 7: Functional Simulation/Verification (Testbench)
- Chapter 8: Synthesis of Combinational and Sequential Logic
- Chapter 9: Post-synthesis design tasks
- Chapter 10: VHDL introduction

Content

A – Overview

- What is structural modeling
- Primitive gates
- Switches
- User-defined primitives

B – Structure model for:

- Combinational Circuit
- Sequential Circuit

Structural model

- When Verilog was first developed (1984) most logic simulators operated on netlists
- Netlist: a list of gates and show how they are connected together
- A natural representation of a digital logic circuit
- Not the most convenient approach to express the test benches

Structural model

- **Structural**

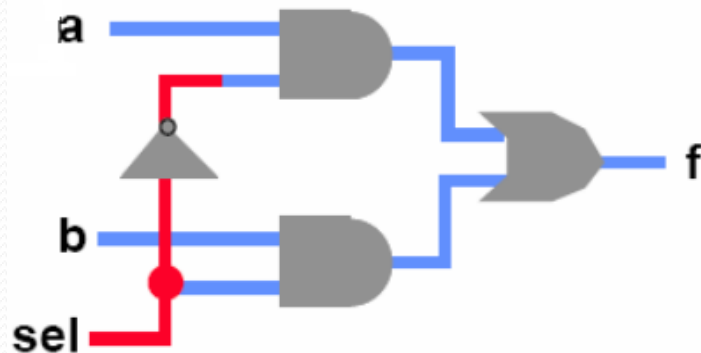
- Explicit structure of the circuit
- How a module is composed as an interconnection of more primitive modules or components
- E.g. Each logic gate initially instantiated and connected to others

- **In Verilog**, a structural model consists of:

- List of connected components
- Like schematics, but **using text**: netlist
- Boring when write, and hard to decode
- Essential without integrated design tools

Structural model

- Structural Models are built from gate primitives, switches, and other modules
- Describe the logic circuit using logic gates



```
module mux (f, a, b, sel);  
    output f;  
    input a, b, sel;  
  
    and #5 g1 (f1, a, nsel),  
            g2 (f2, b, sel);  
    or #5 g3 (f, f1, f2);  
    not g4 (nsel, sel);  
endmodule
```

Primitive gates

- 12 primitive logic gates predefined in the Verilog HDL

and	nand	logical AND/NAND
or	nor	logical OR/NOR
xor	xnor	logical XOR/XNOR
buf	not	buffer/inverter
bufif0	notif0	Tristate with low enable
bifif1	notif1	Tristate with high enable

- Advantages:**

- ✓ Gates provide a much closer one-to-one mapping between the actual circuit and the model.
- ✓ There is no continuous assignment equivalent to the bidirectional transfer gate.

Primitive gates

And/Or/Nand/Nor/Xor/Xnor

One scalar output

Multiple scalar inputs

The first terminal in the list of gate terminals is an output and the other terminals are inputs

Terminal list



```
wire OUT, IN1, IN2; // basic gate instantiations
```

```
and a1(OUT, IN1, IN2);
```

```
nand na1(OUT, IN1, IN2);
```

```
or or1(OUT, IN1, IN2);
```

```
nor nor1(OUT, IN1, IN2);
```

```
xor x1(OUT, IN1, IN2);
```

```
xnor nx1(OUT, IN1, IN2);
```



and



nand



or



nor



xor



xnor

Verilog automatically instantiates the appropriate gate.



```
// More than two inputs; 3 input nand gate
```

```
nand na1_3inp(OUT, IN1, IN2, IN3);
```

```
// gate instantiation without instance name
```

```
and (OUT, IN1, IN2); // legal gate instantiation
```

Primitive gates

Buf/Not Gates

One scalar input

One or more scalar outputs

The last terminal in the port list is connected to the input

// basic gate instantiations.

```
buf b1(OUT1, IN);
```

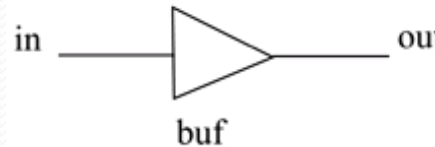
```
not n1(OUT1, IN);
```

// More than two outputs

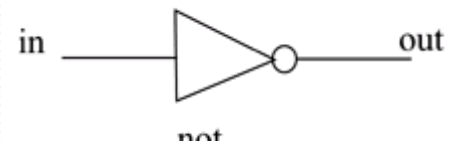
```
buf b1_2out(OUT1, OUT2, IN);
```

// gate instantiation without instance name

```
not (OUT1, IN); // legal gate instantiation
```



buf	in	out
	0	0
	1	1
	x	x
	z	x



not	in	out
	0	1
	1	0
	x	x
	z	x

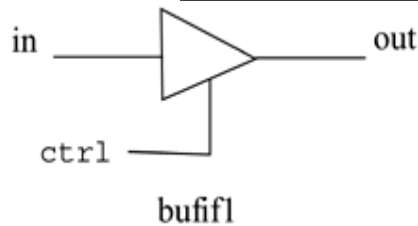
Primitive gates

Bufif/notif

Gates with an additional control signal on **buf** and **not** gates

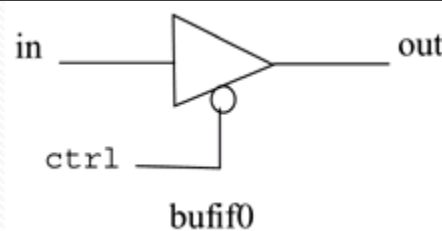
Propagate only if control signal is asserted.

Propagate z if their control signal is de-asserted



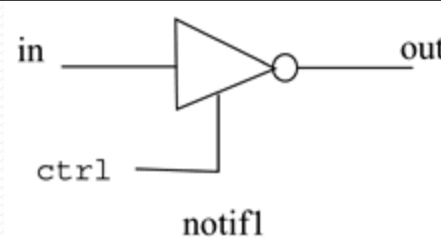
		ctrl			
bufif1		0	1	x	z
in	0	z	0	L	L
	1	z	1	H	H
	x	z	x	x	x
	z	z	x	x	x

bufif1 b1 (out, in, ctrl);



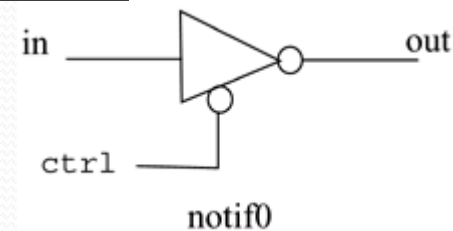
		ctrl			
bufif0		0	1	x	z
in	0	0	z	L	L
	1	1	z	H	H
	x	x	z	x	x
	z	x	z	x	x

bufif0 b0 (out, in, ctrl);



		ctrl			
notif1		0	1	x	z
in	0	z	1	H	H
	1	z	0	L	L
	x	z	x	x	x
	z	z	x	x	x

notif1 n1 (out, in, ctrl);



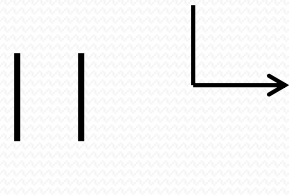
		ctrl			
notif0		0	1	x	z
in	0	1	z	H	H
	1	0	z	L	L
	x	x	z	x	x
	z	x	z	x	x

notif0 n0 (out, in, ctrl);

Primitive gates

➤ Array of Instances

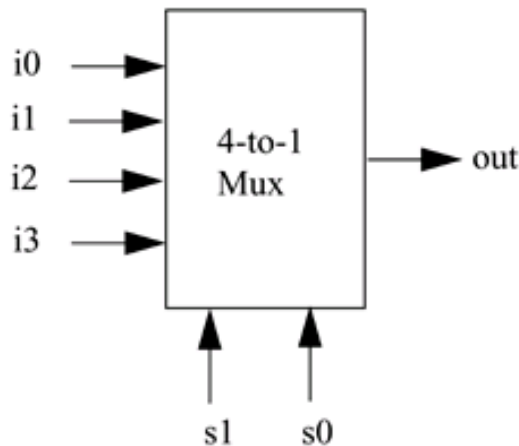
```
wire [7:0] OUT, IN1, IN2;  
nand n_gate[7:0](OUT, IN1, IN2);
```

 The instances differ from each other only by the index of the vector to which they are connected

```
// This is equivalent to the following 8 instantiations  
nand n_gate0(OUT[0], IN1[0], IN2[0]);  
nand n_gate1(OUT[1], IN1[1], IN2[1]);  
nand n_gate2(OUT[2], IN1[2], IN2[2]);  
nand n_gate3(OUT[3], IN1[3], IN2[3]);  
nand n_gate4(OUT[4], IN1[4], IN2[4]);  
nand n_gate5(OUT[5], IN1[5], IN2[5]);  
nand n_gate6(OUT[6], IN1[6], IN2[6]);  
nand n_gate7(OUT[7], IN1[7], IN2[7]);
```

Primitive gates

➤ Example: Gate-level multiplexer



s1	s0	out
0	0	I0
0	1	I1
1	0	I2
1	1	I3

4-to-1 Multiplexer

```
// Module 4-to-1 multiplexer.
```

```
// Port list is taken exactly from the I/O diagram.
```

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
```

```
// Port declarations from the I/O diagram
```

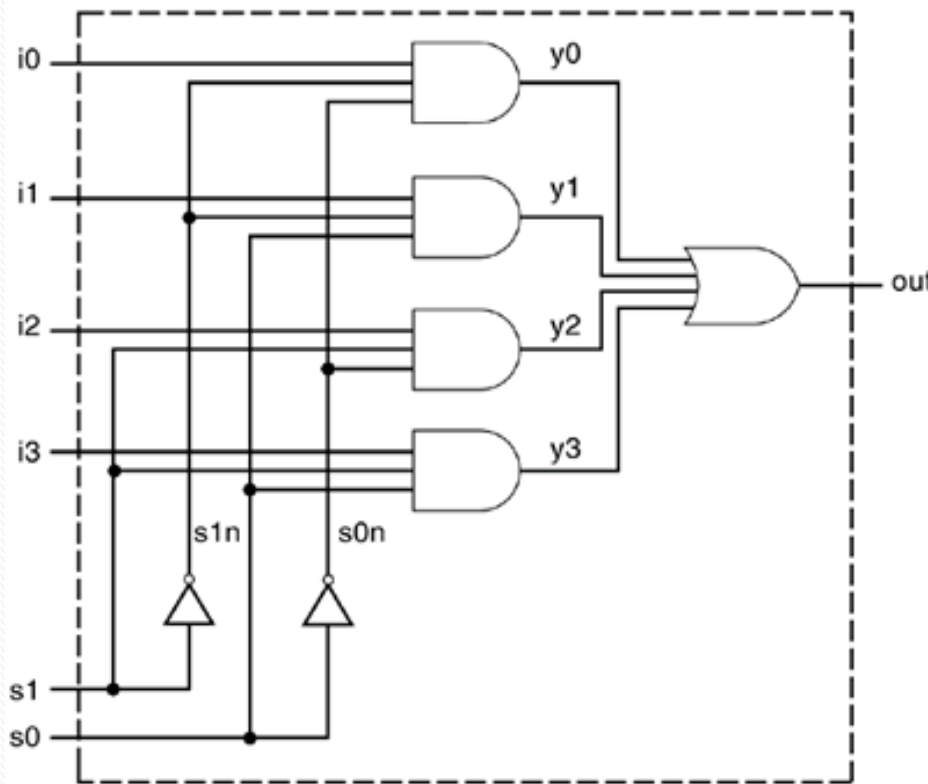
```
output out;
```

```
input i0, i1, i2, i3;
```

```
input s1, s0;
```

Primitive gates

➤ Example: Gate-level multiplexer



Logic Diagram for 4-to-1 Multiplexer

```
// Internal wire declarations
wire s1n, s0n;
wire y0, y1, y2, y3;
// Gate instantiations
// Create s1n and s0n signals.
not (s1n, s1);
not (s0n, s0);
// 3-input and gates instantiated
and (y0, i0, s1n, s0n);
and (y1, i1, s1n, s0);
and (y2, i2, s1, s0n);
and (y3, i3, s1, s0);
// 4-input or gate instantiated
or (out, y0, y1, y2, y3);
endmodule
```

Switches

- There are two kinds of switch:
 - * **MOS switches** :
cmos, nmos, pmos, rcmos, rnmos, rpmos
 - * **Bidirectional pass switches**:
tran, rtran, tranif1, rtranif1, tranif0, rtranif0
- **Advantages:**
 - Gates provide a much closer one-to-one mapping between the actual circuit and the model.
 - There is no continuous assignment equivalent to the bidirectional transfer gate.

Switches

MOS switches: nmos, pmos, rnoms, rpmos

Unidirectional channel for data, similar to **bufif** gate

nmos

rnmos

when "off"

nmos

when "on"

rnmos

when "on"

R

R

CONTROL

DATA

Ex: nmos n1 (out, data, control)

pmos

rpmos

when "off"

pmos

when "on"

rpmos

when "on"

R

R

CONTROL

DATA

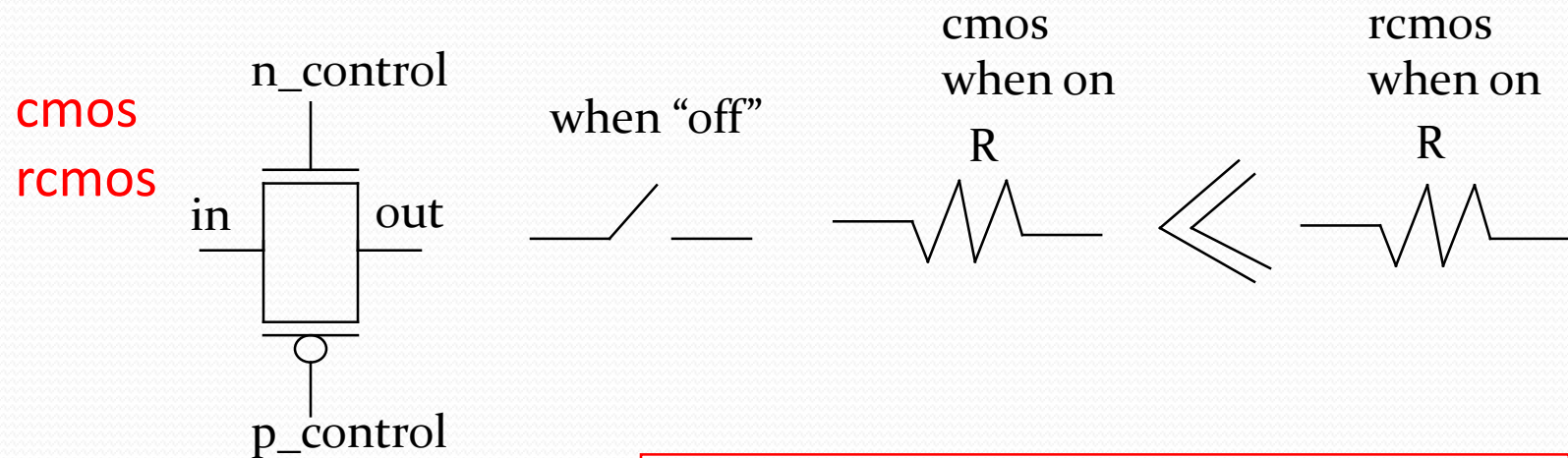
Ex: pmos p1 (out, data, control)

nmos rnmos	CONTROL				
		0	1	x	z
D	0	z	0	L	L
A	1	z	1	H	H
T	x	z	x	x	x
A	z	z	z	z	z

pmos rpmos	CONTROL				
		0	1	x	z
D	0	0	z	L	L
A	1	1	z	H	H
T	x	x	z	x	x
A	z	z	z	z	z

Switches

MOS Switches: cmos, rcmos



cmos (out, in, n_control, p_control)

Switches

Bidirectional pass switches

- **tranif0, tranif1, rtranif0, rtranif1**: block signal when turn off, pass signal when turn on

Ex: tranif0 (inout1, inout2, control)

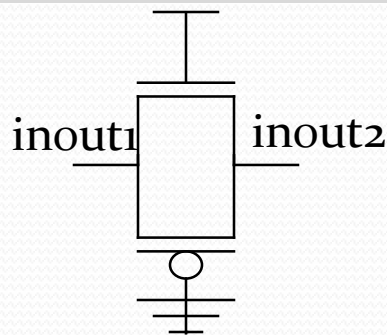
- **tran, rtran**: always pass signal

Ex: tran (inout1, inout2)

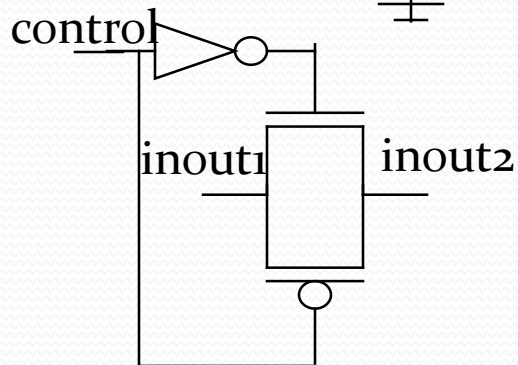
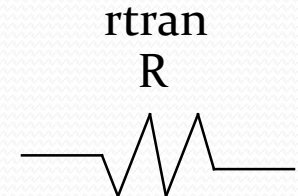
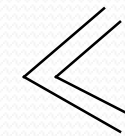
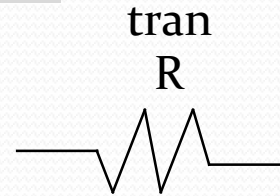
- Terminals be scalar nets or bit-select of vector nets

Switches

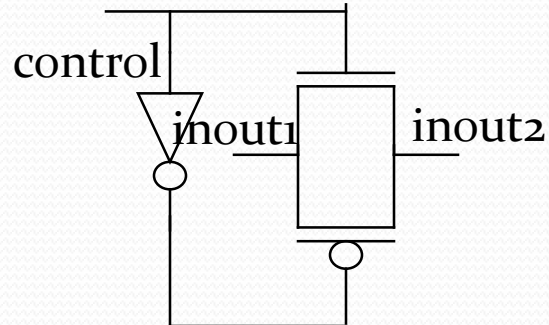
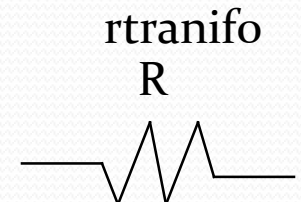
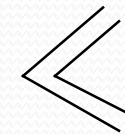
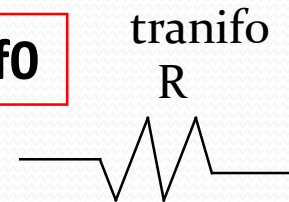
Bidirectional pass switches



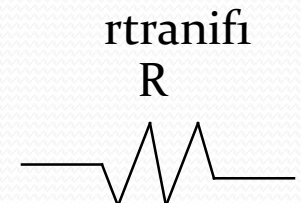
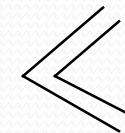
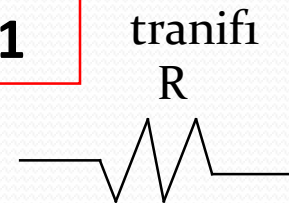
tran, rtran



tranif0, rtranif0



tranif1, rtranif1



Switches

- Ref “*Verilog digital system design*”, Zainalabedin Navabi for design examples at switch level

pull gates

- A **pullup** source shall place a logic value 1 on the nets connected in its terminal list.
- A **pulldown** source shall place a logic value 0 on the nets connected in its terminal list.

Ex: **pullup** (strong1) p1 (neta), p2 (netb);

Strength modeling

- Allows specification of drive strength for primitive **gate outputs** and **nets**.
- **Gate output** or **net** signal strength values are specified in a set of parenthesis that include a strength value for logic **0** and one for logic **1**.
- Drive strengths for logic **0** (*strength0*):
supply0, strong0, pull0, weak0, highz0
- Drive strengths for logic **1** (*strength1*):
supply1, strong1, pull1, weak1, highz1
- Charge strengths, representing the strength of a capacitive **net**:
large, medium, small

Strength modeling

Drive strength values of primitive gate outputs

	LOGIC GATES	PULL GATES		SWITCHES
		Pullup	Pulldown	
<i>Strength0</i>	supply0 strong0 pull0 weak0 highz0		supply0 strong0 pull0 weak0	No strength
<i>Strength1</i>	supply1 strong1 pull1 weak1 highz1	supply1 strong1 pull1 weak1		No strength

Ex: `nand (pull0, pull1) # (3, 5) n1 (w, a, b, c);`

Strength modeling

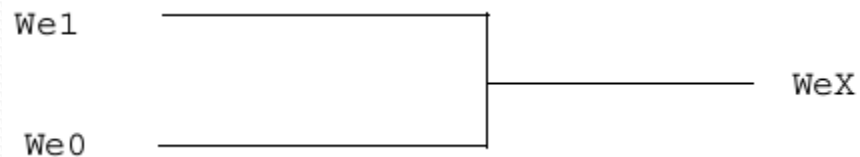
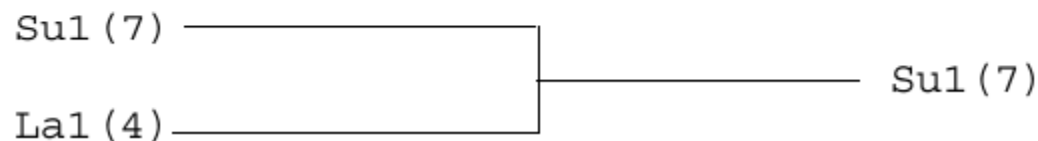
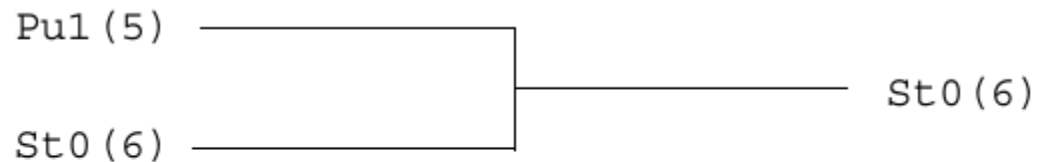
Strength values of nets

	<i>wire (tri), wand (triand), wor (trior), tri0, tri1</i>	<i>Level</i>	<i>trireg</i>	
	<i>Strength values</i>		<i>Strength values</i>	
<i>Strength0</i>	supply0	7		
	strong0	6		
	pull0	5		
		4	large (0)	
	weak0	3		
		2	medium (0)	
		1	small (0)	
<i>Strength1</i>	highz0	0		
	highz1	0		
		1	small (1)	
		2	medium (1)	
	weak1	3		
		4	large (1)	
	pull1	5		
	strong1	6		
	supply1	7		
	Drive strength		Charge strength	

Strength modeling

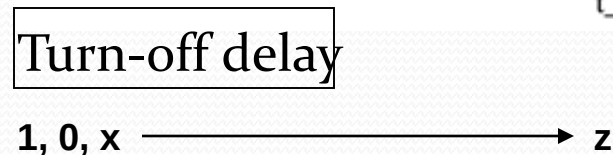
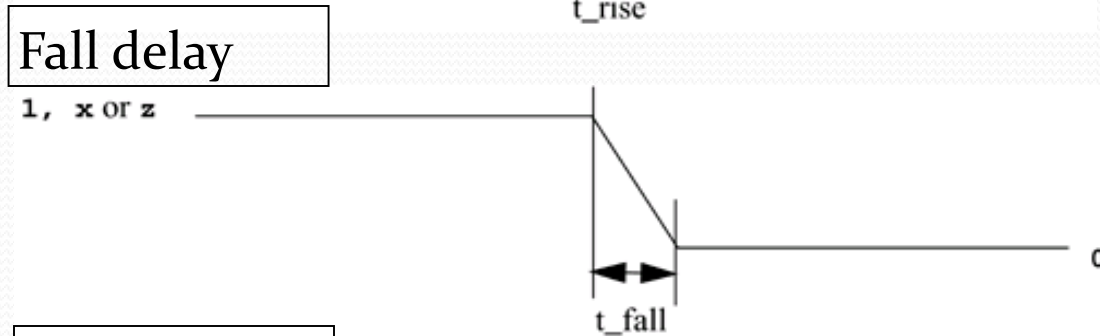
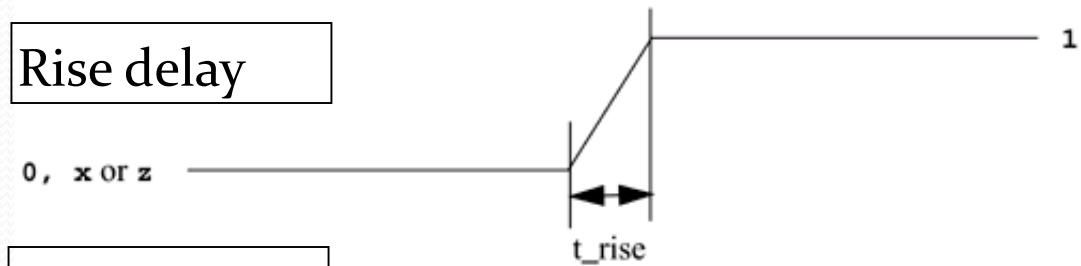
Strength level

The stronger signal shall dominate all the weaker drivers and determine the result.



Gate & Net delays

- Rise, Fall, and Turn-off Delays



Note: If a value changes to **x**, the minimum of the three delays is considered

Gate & Net delays

- Rise, Fall, and Turn-off Delays

- If only **one delay** is specified, this value is used for all transitions.
- If **two delays** are specified, they refer to the rise and fall delay values.
The turn-off delay is the minimum of the two.
- If all **three delays** are specified, they refer to rise, fall, and turn-off delay values.
- If **no delays** are specified, the default value is zero.

Example:

```
and #(5) a1(out, i1, i2); //Delay of 5 for all transitions
```

```
and #(4,6) a2(out, i1, i2); // Rise = 4, Fall = 6
```

```
bufif0 #(3,4,5) b1 (out, in, control); // Rise = 3, Fall = 4, Turn-off = 5
```

Gate & Net delays

- **Min/Typ/Max values**

For each type of delay three values, ***min/typ/max***, can be specified.

Min/typ/max values: model devices whose delays vary within a [min max] range because of the IC fabrication process variations.

min	The minimum delay value that the designer expects the gate to have
typ	The typical delay value that the designer expects the gate to have
max	The maximum delay value that the designer expects the gate to have

Min, typ, or max values can be chosen at Verilog run time.

Method of choosing a ***min/typ/max*** value may vary for different simulators or operating systems

Gate & Net delays

- Min/Typ/Max values

```
and #(4:5:6) a1(out, i1, i2);  
// One delay  
// mindelays, delay= 4  
// typdelays, delay= 5  
// maxdelays, delay= 6  
  
and #(3:4:5, 5:6:7) a2(out, i1, i2);  
// Two delays  
// mindelays, rise= 3, fall= 5, turn-off = min(3,5)  
// typdelays, rise= 4, fall= 6, turn-off = min(4,6)  
// maxdelays, rise= 5, fall= 7, turn-off = min(5,7)  
  
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1,i2);  
// Three delays  
// mindelays, rise= 2 fall= 3 turn-off = 4  
// typdelays, rise= 3 fall= 4 turn-off = 5  
// maxdelays, rise= 4 fall= 5 turn-off = 6
```

Gate & Net delays

Ex:

```
buf          b1(a, b);    // Zero delay
buf #3       b2(c, d);    // Delay of 3
buf #(4,5)   b3(e, f);    // Rise=4, fall=5
buf #(3:4:5) b4(g, h);    // Min-typ-max
```

Built-in Gate & Switch

n_input gates	n_output gates	Three-state gates	Pull gates	MOS switches	Bidirectional switches
and	buf	bufif0	pulldown	cmos	rtran
nand	not	bufif1	pullup	nmos	rtranif0
nor		notif0		pmos	rtranif1
or		notif1		remos	tran
xnor				rnmos	tranif0
xor				rpmos	tranif1

A gate/switch instance declaration consists of:

- The keyword that names the type of gate or switch primitive
- An optional drive strength
- An optional propagation delay
- An optional identifier that names each gate or switch instance
- An optional range for array of instances
- The terminal connection list

Ex: **nand** (pull0, pull1) #(3, 5) n1 [7:0] (w, a, b, c)

User-Defined Primitives

- The set of predefined gate primitives by designing and specifying new primitive elements
- Instances of these new UDPs can be used in exactly the same manner as the gate primitives
- Way to define combinational and sequential elements using a truth table
- Each UDP has exactly one output, which can be in one of three states: 0, 1, or x.
- Often simulate faster than using expressions, collections of primitive gates, etc.
- Gives more control over behavior with X inputs
- Most often used for specifying custom gate libraries

Example: Combinational UDPs

primitive multiplexer (mux, control, dataA, dataB);

output mux;

input control, dataA, dataB;

table

// control dataA dataB mux

0 1 0 : 1 ;

0 1 1 : 1 ;

0 1 x : 1 ;

0 0 0 : 0 ;

0 0 1 : 0 ;

0 0 x : 0 ;

1 0 1 : 1 ;

1 1 1 : 1 ;

1 x 1 : 1 ;

1 0 0 : 0 ;

1 1 0 : 0 ;

1 x 0 : 0 ;

x 0 0 : 0 ;

x 1 1 : 1 ;

endtable

endprimitive

output mux;

input control, dataA, dataB;

table

// control dataA dataB mux

0 1 ? : 1 ; // ? = 0 1 x

0 0 ? : 0 ;

1 ? 1 : 1 ;

1 ? 0 : 0 ;

x 0 0 : 0 ;

x 1 1 : 1 ;

endtable

endprimitive

User-Defined Primitives

Example: Level-sensitive sequential UDPs

```
primitive latch (q, clock, data);  
output q; reg q;  
input clock, data;  
table  
  // clock data q q+  
  0  1 : ? : 1 ;  
  0  0 : ? : 0 ;  
  1  ? : ? : - ; // - = no change  
endtable  
endprimitive
```

User-Defined Primitives

Example: Edge-sensitive sequential UDPs

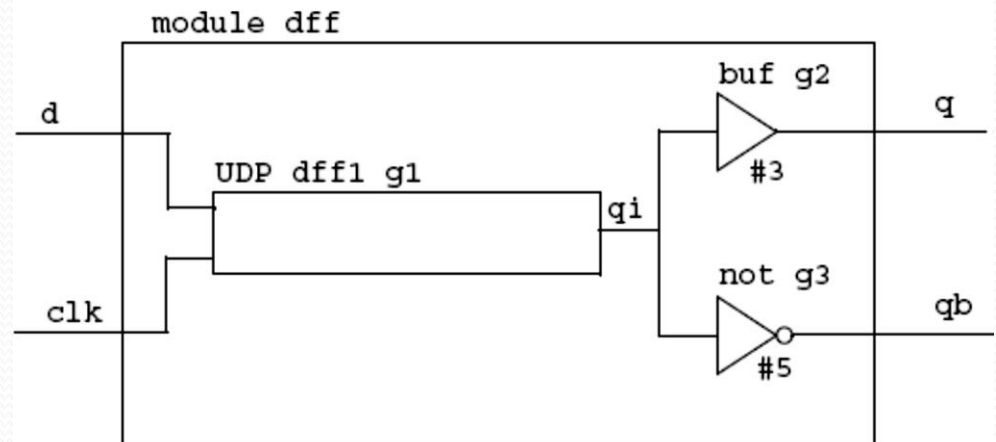
```
primitive d_edge_ff (q, clock, data);  
output q; reg q;  
input clock, data;  
table  
  // obtain output on rising edge of clock  
  //  clock data q q+  
    (01) 0 : ? : 0 ;  
    (01) 1 : ? : 1 ;  
    (0?) 1 : 1 : 1 ;  
    (0?) 0 : 0 : 0 ;  
  // ignore negative edge of clock  
    (?0) ? : ? : - ;  
  // ignore data changes on steady clock  
    ?   ? : ? : - ; // - = no change  
endtable  
endprimitive
```

User-Defined Primitives

Initial statement in UDPs and instances

```
primitive dff1 (q, clk, d);  
input clk, d;  
output q; reg q;  
initial q = 1'b1;  
table  
// clk d q q+  
r 0 : ? : 0 ;  
r 1 : ? : 1 ;  
f ? : ? : - ;  
? * : ? : - ;  
endtable  
endprimitive
```

```
module dff (q, qb, clk, d);  
input clk, d;  
output q, qb;  
dff1 g1 (qi, clk, d);  
buf #3 g2 (q, qi);  
not #5 g3 (qb, qi);  
endmodule
```



Content

A – Overview

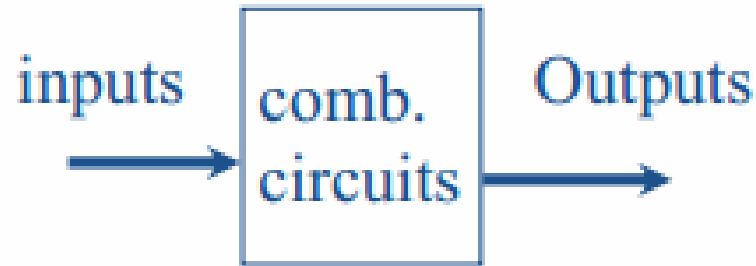
- What is structural modeling
- Primitive gates
- Switches
- User-defined primitives

B – Structure model for:

- Combinational Circuit
- Sequential Circuit

Combinational Circuit

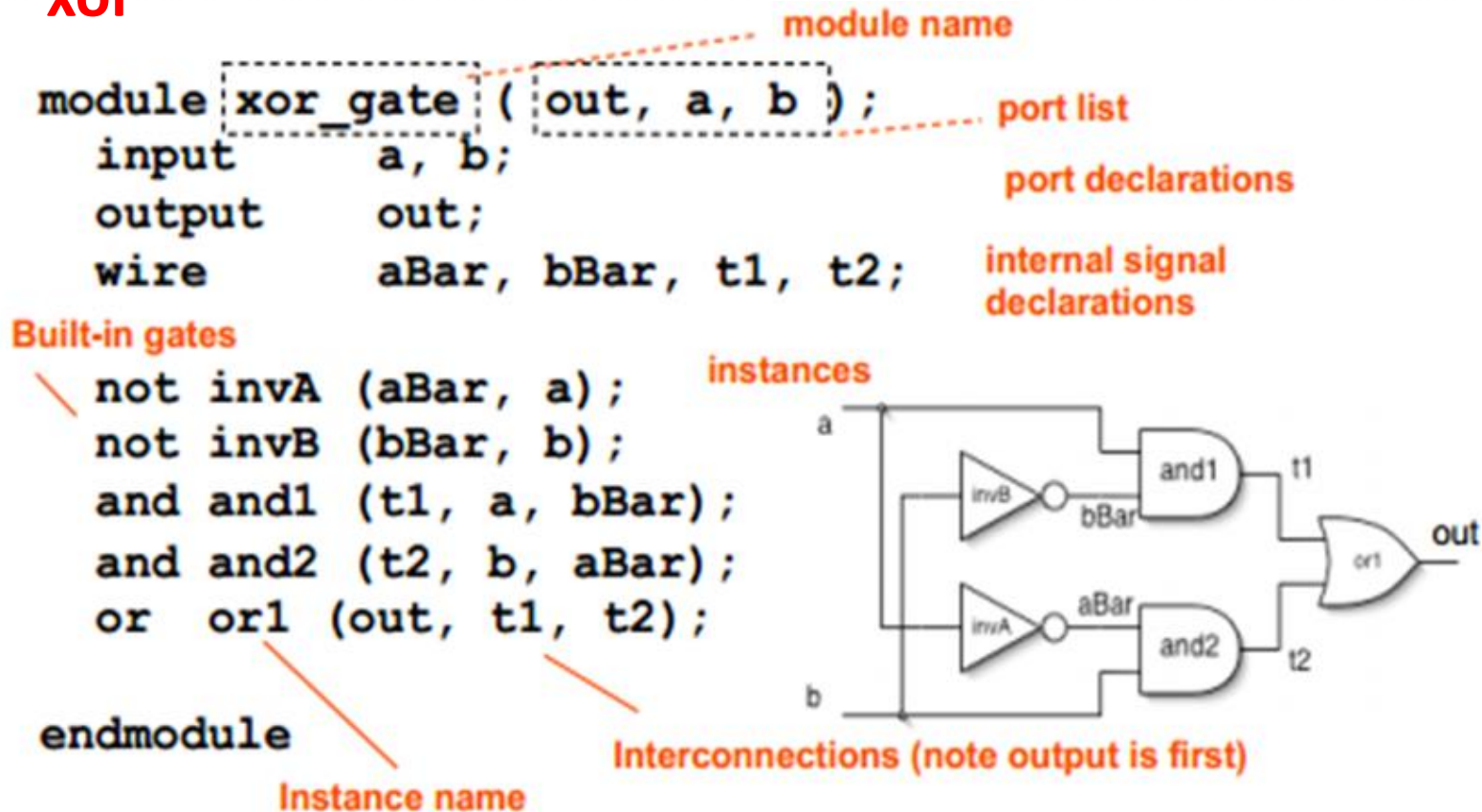
- Outputs are functions of the current inputs
- Logic without state variables
- No clock involved



- **Examples**
 - multiplexer, demultiplexers
 - decoder
 - encoder
 - adder

Example: xor

xor

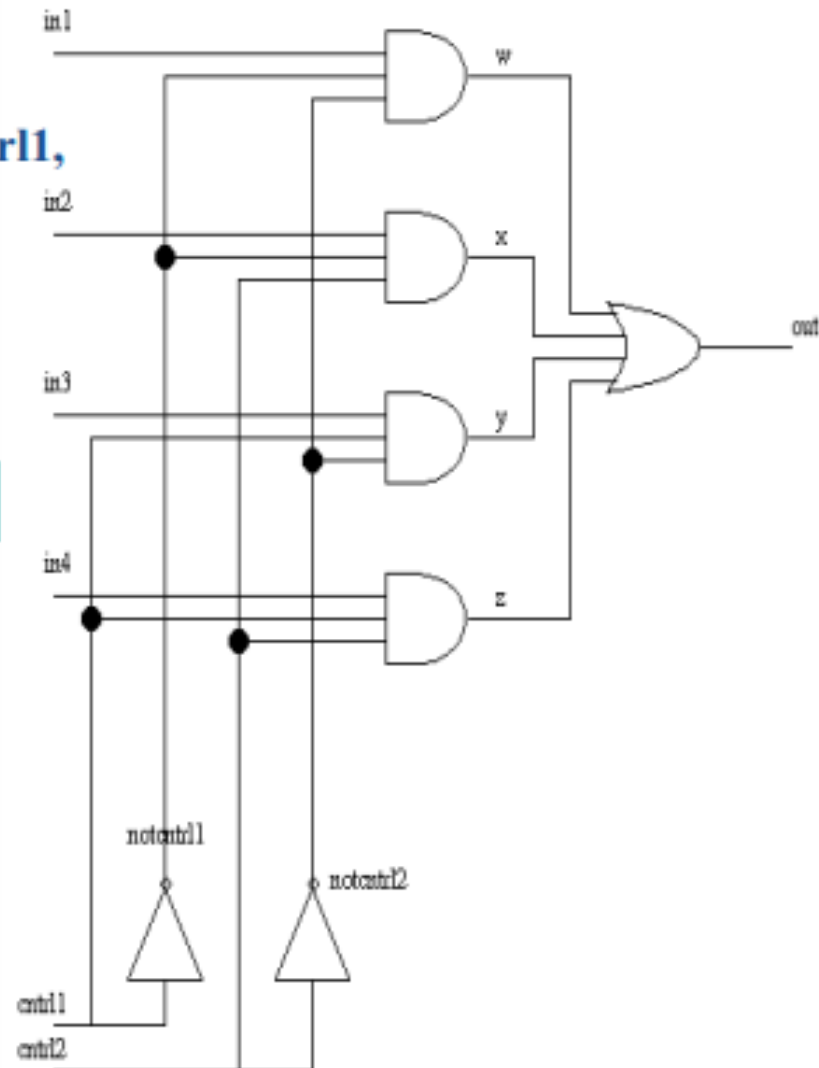


- The instantiated gates are always active
- xor gate already exists as a built-in (so really no need to define it)

Example: mux 4-to-1

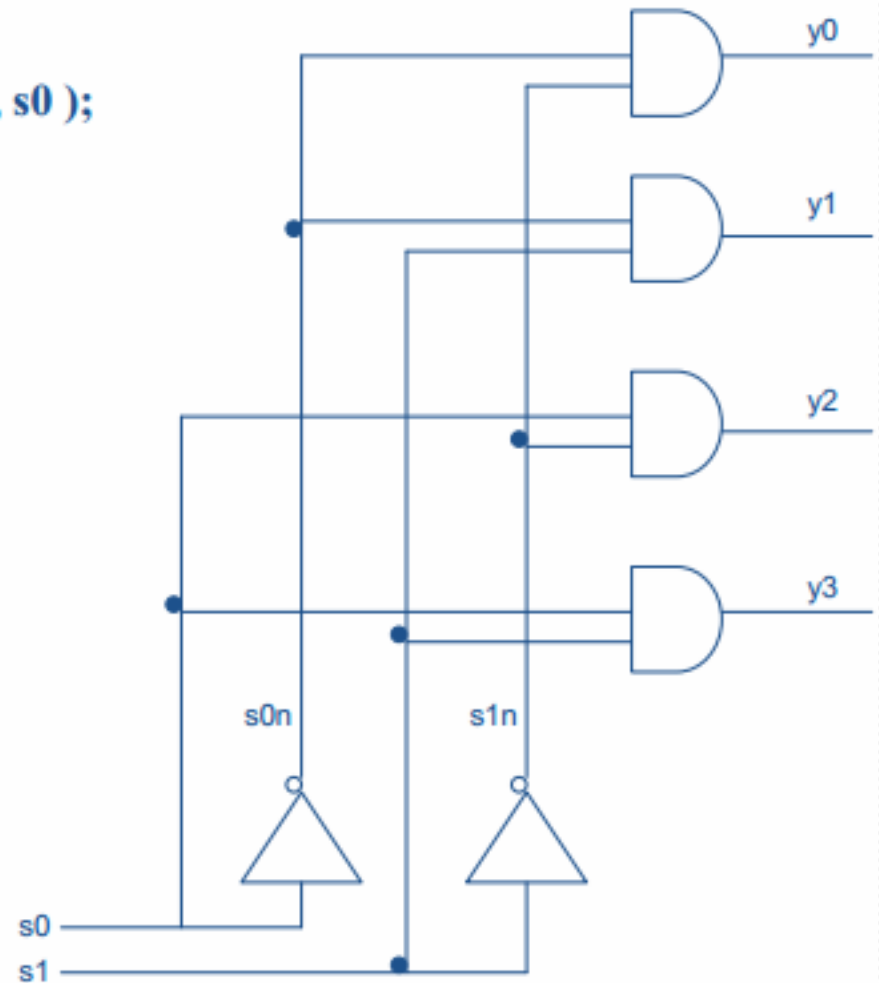
```
module multiplexor4_1(out, in1, in2, in3, in4, cntrl1,  
cntrl2);  
output out;  
input in1, in2, in3, in4, cntrl1, cntrl2;  
wire notcntrl1, notcntrl2, w, x, y, z;  
  
not (notcntrl1, cntrl1);  
not (notcntrl2, cntrl2);  
and (w, in1, notcntrl1, notcntrl2);  
and (x, in2, notcntrl1, cntrl2);  
and (y, in3, cntrl1, notcntrl2);  
and (z, in4, cntrl1, cntrl2);  
or (out, w, x, y, z);  
endmodule
```

Recall default type is wire.

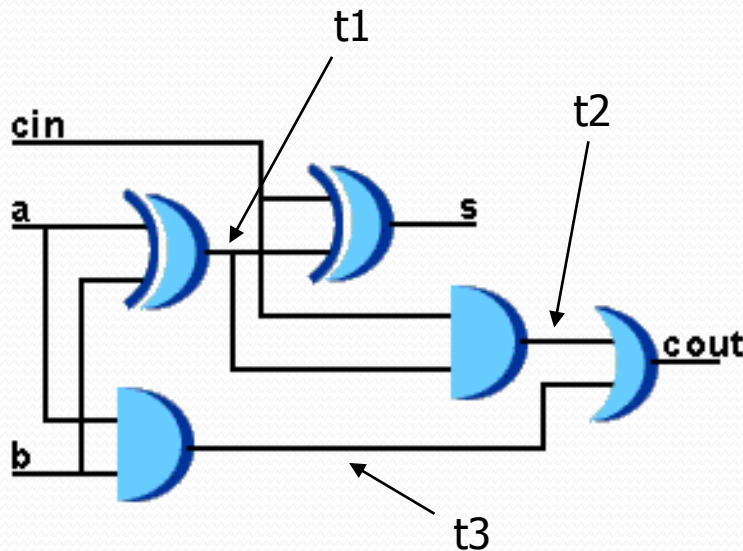


Example: 2-to-4 decoder

```
module decoder2_to_4 ( y0, y1, y2, y3, s1, s0 );  
  // Inputs and outputs  
  output y0, y1, y2, y3;  
  input s1, s0;  
  // Internal wires  
  wire s1n, s0n;  
  // Create complements of s1 and s0  
  not ( s1n, s1 );  
  not ( s0n, s0 );  
  and ( y0, s1n, s0n );  
  and ( y1, s1n, s0 );  
  and ( y2, s1, s0n );  
  and ( y3, s1, s0 );  
  
endmodule
```



Example: Full adder 1-bit



Full adder 1-bit

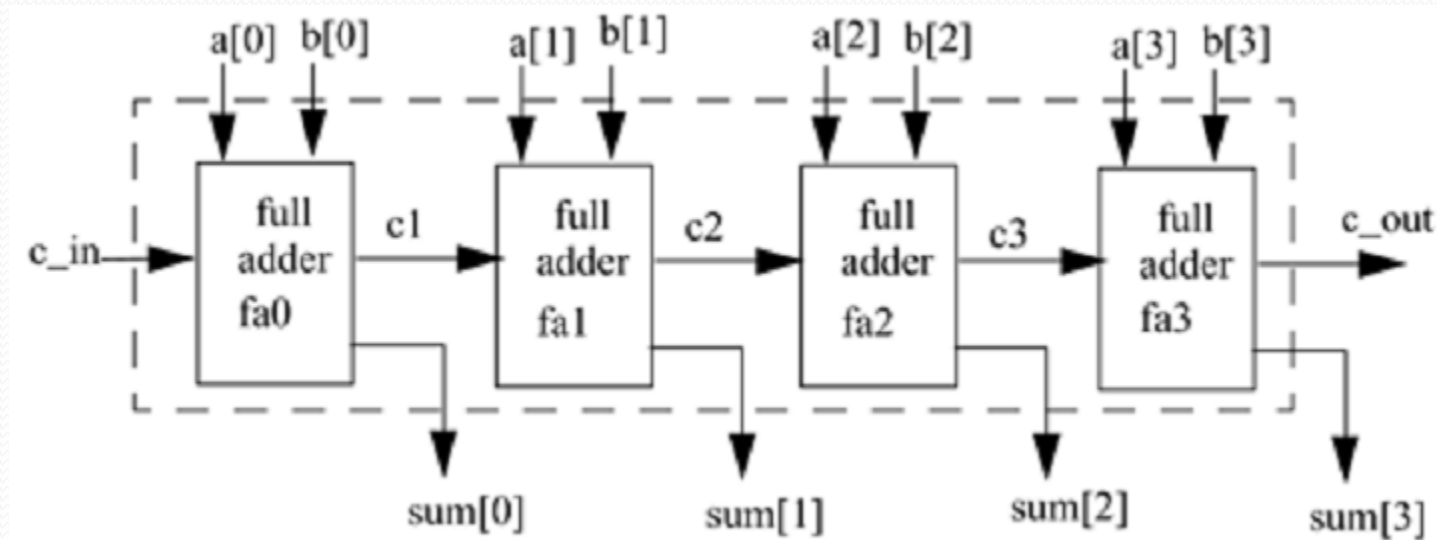
```
// Define a 1-bit full adder
module fulladd(sum, c_out, a, b, c_in);

// I/O port declarations
output sum, c_out;
input a, b, c_in;

// Internal nets
wire s1, c1, c2;

// Instantiate logic gate primitives
xor (s1, a, b);
and (c1, a, b);
xor (sum, s1, c_in);
and (c2, s1, c_in);
xor (c_out, c2, c1);
endmodule
```

Example: 4-bit ripple carry full adder



Example: 4-bit ripple carry full adder

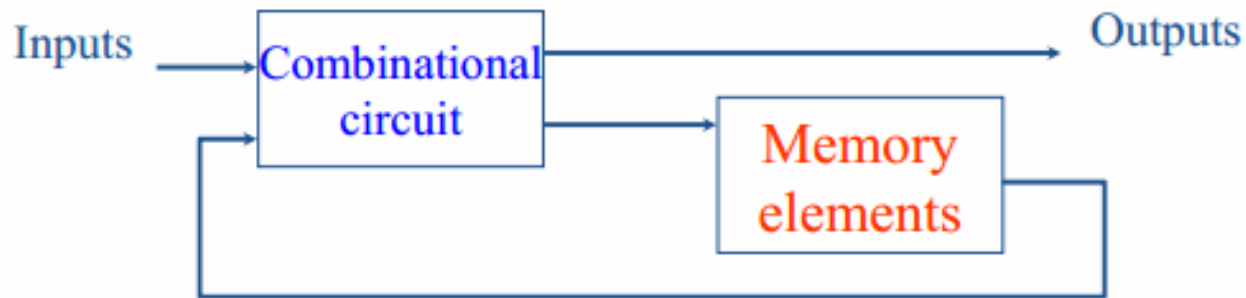
```
// Define a 4-bit full adder
module fulladd4(sum, c_out, a, b, c_in);
// I/O port declarations
output [3:0] sum;
output c_out;
input [3:0] a, b;
input c_in;

// Internal nets
wire c1, c2, c3;

// Instantiate four 1-bit full adders.
fulladd fa0(sum[0], c1, a[0], b[0], c_in);
fulladd fa1(sum[1], c2, a[1], b[1], c1);
fulladd fa2(sum[2], c3, a[2], b[2], c2);
fulladd fa3(sum[3], c_out, a[3], b[3], c3);
endmodule
```

Sequential Circuit

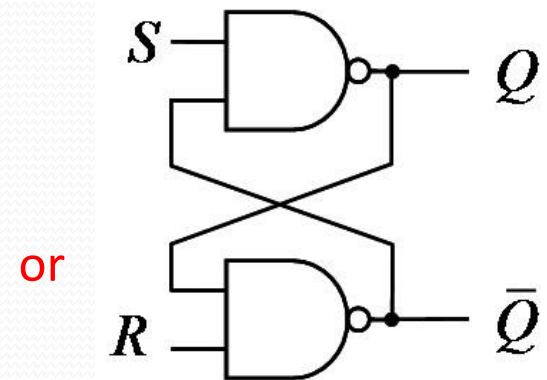
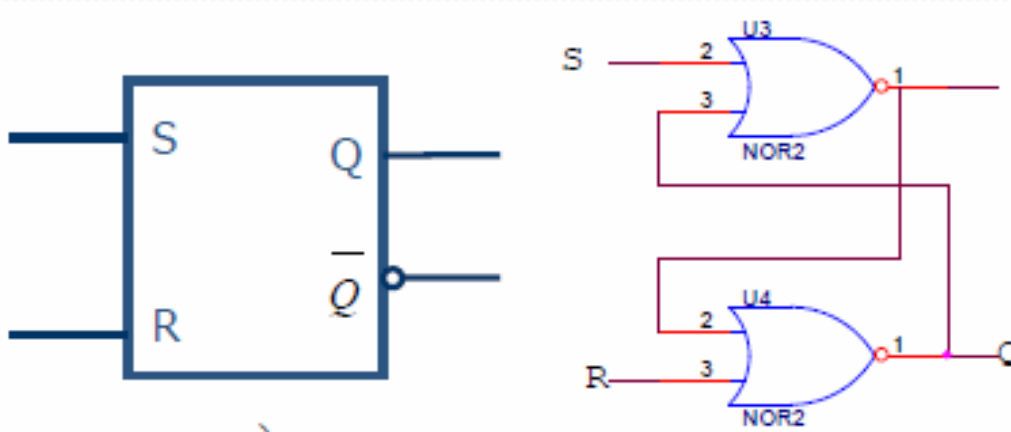
- A feedback path
- The state of the sequential circuits
- The state transition
 - synchronous circuits
 - asynchronous circuits



Example: Latch, Flip-flop, Register, Counter, State machine, Processor...

Example: SR latch

- Set-Reset (SR) Latch :



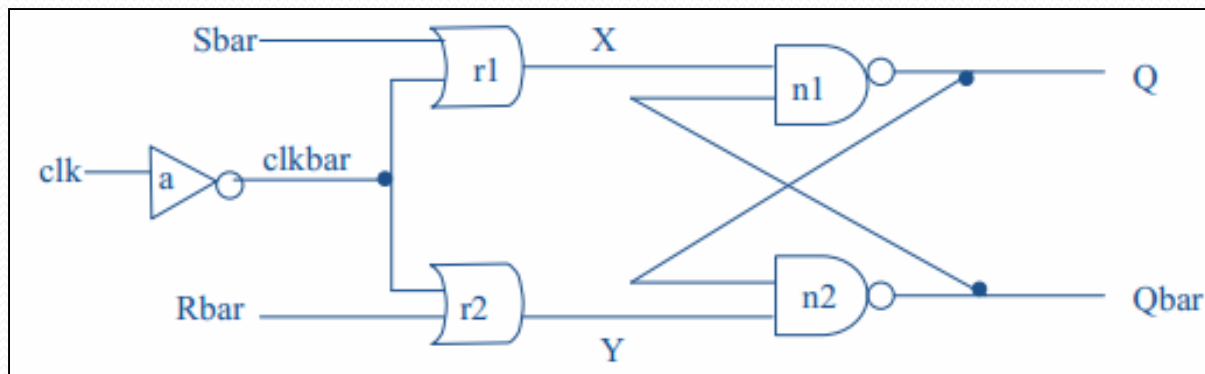
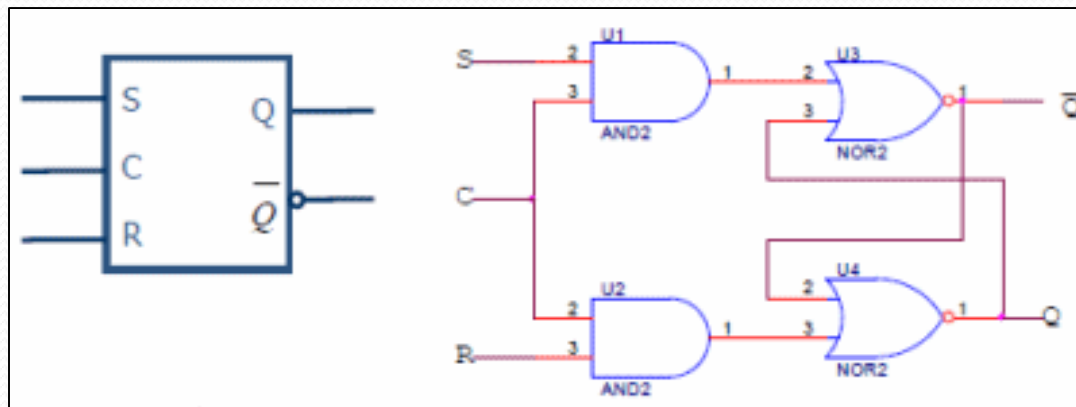
S	R	Q(t+1)	
0	0	Q(t)	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	X	Indeterminate

Example: SR latch

- Clocked Set-Reset (SR) Latch :

(1) State can change only **when clock is high**

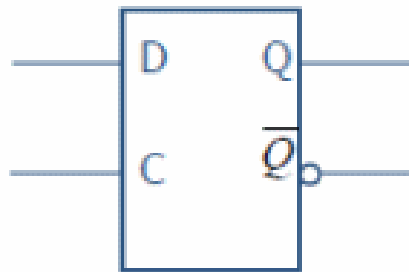
(2) Potential non-deterministic behavior if both input Sbar and Rbar are 0



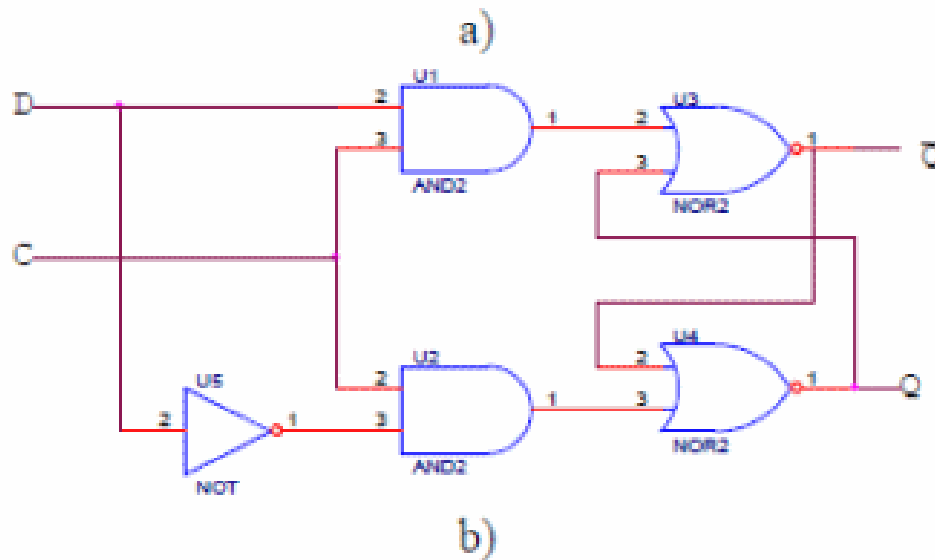
Example: SR latch

```
module clockedSR_latch(Q, Qbar, Sbar, Rbar, clk);  
  // Port declarations  
  output Q, Qbar;  
  input Sbar, Rbar, clkbar;  
  wire X, Y ;  
  // Gate declarations  
  not a(clkbar, clk);  
  or r1(X, Sbar, clkbar);  
  or r2(Y, Rbar, clkbar);  
  nand n1(Q, X, Qbar);  
  nand n2(Qbar, Y, Q);  
endmodule
```

D flip-flop

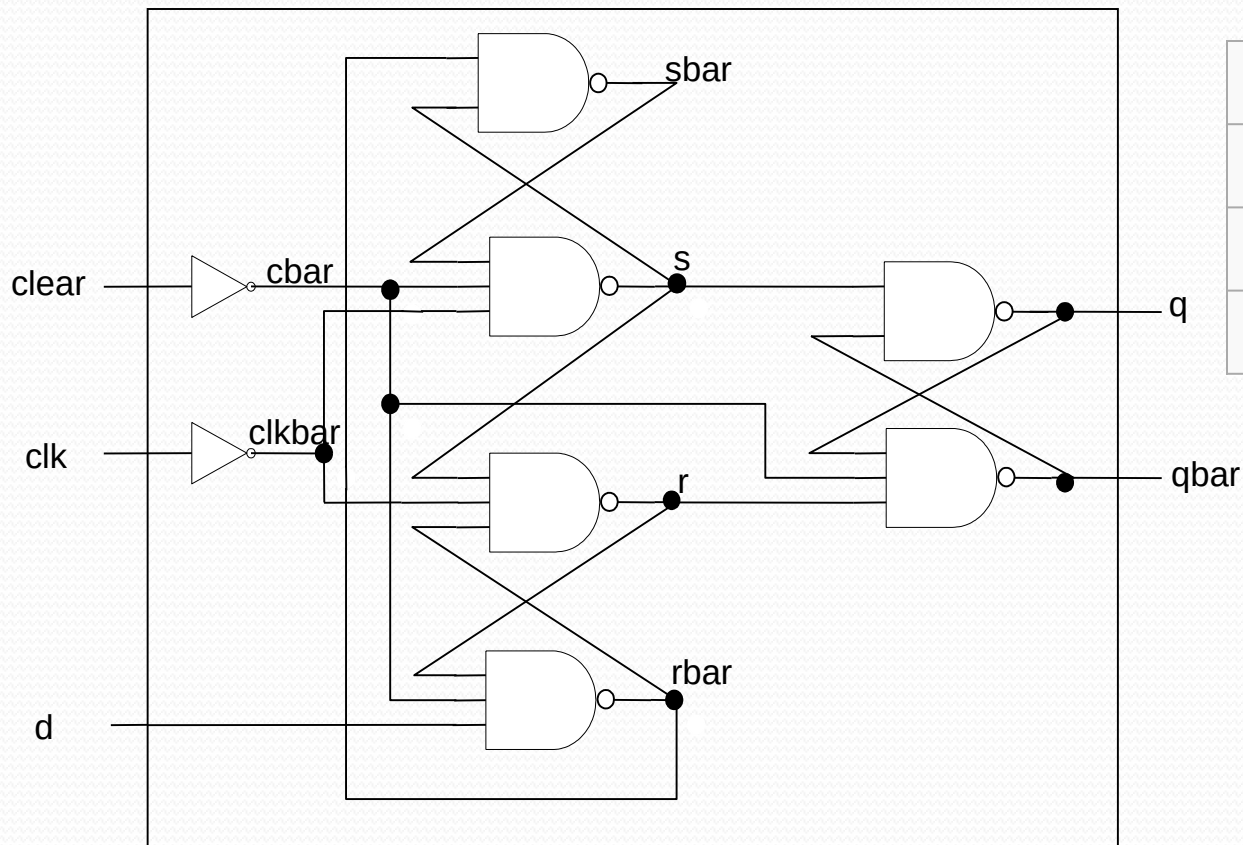


D	$Q(t+1)$
0	0 Clear to 0
1	1 Set to 1



Example: D flip-flop

■ Negative edge-triggered D flip-flop



Clock	D	Q_{next}
Falling edge	0	0
Falling edge	1	1
Non-falling	X	Q

Negative edge-triggered D-flipflop implemented using 3 SR latches

Example: D flip-flop

Negative edge-triggered D flip-flop

```
module edge_dff(q, qbar, d, clk, clear);  
  output q,qbar;  
  input d, clk, clear;  
  wire s, sbar, r, rbar,cbar;  
  not (cbar, clear);  
  not (clkbar, clk);  
  
  // Input latches  
  nand (sbar, rbar, s);  
  nand (s, sbar, cbar, clkbar);  
  nand (r, rbar, clkbar, s);  
  nand (rbar, r, cbar, d);  
  
  // Output latch  
  nand (q, s, qbar);  
  nand (qbar, q, r, cbar);  
endmodule
```

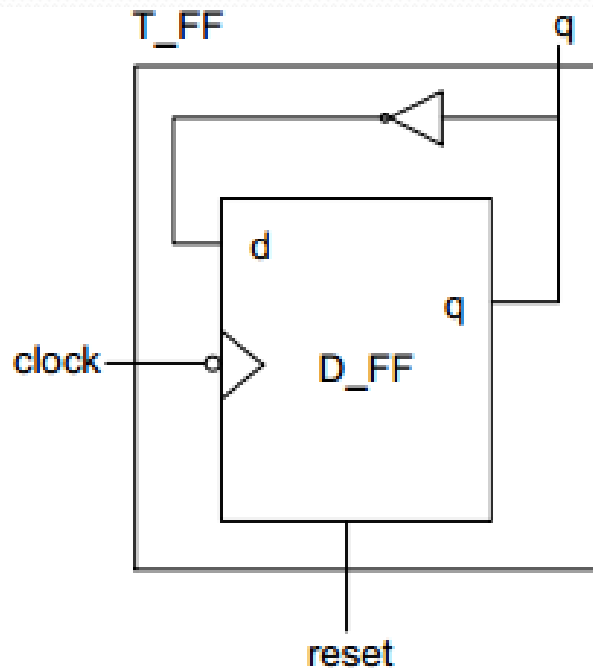
Gate-level D-flipflop

- (1) Negative edge-triggered
- (2) Made from 3 SR-latches (see circuit)

Extremely important module – it is the fundamental unit of computer memory!

Example: T flip-flop

■ T (Toggle) flip-flop

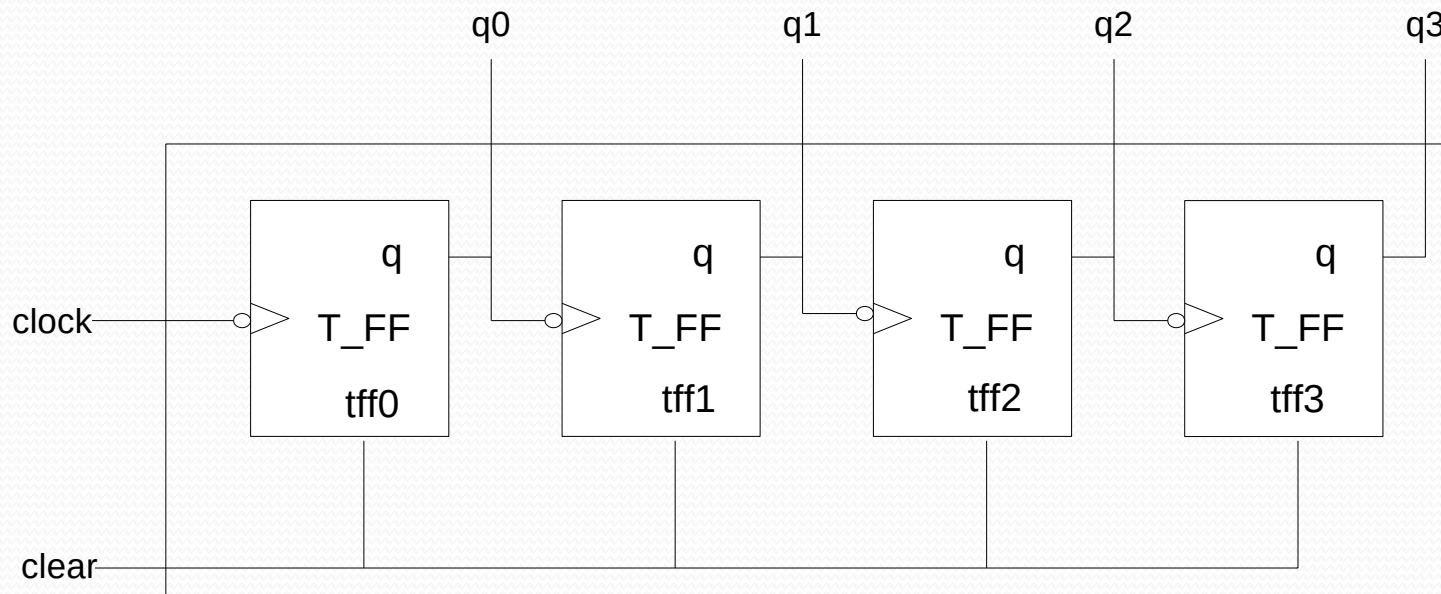


```
module t_ff(q, clk, clear);  
  output q;  
  input clk, clear;  
  
  // Instantiate the edge triggered DFF  
  // Complement of output q is fed back.  
  // Notice qbar not needed. Empty port.  
  edge_dff ff1(q, ~q, clk, clear);  
  
endmodule
```

Negative edge-triggered T-flipflop implemented using a D-flipflop and an inverter gate – *toggles* every clock cycle

Example: Counter

■ 4-bit ripple counter



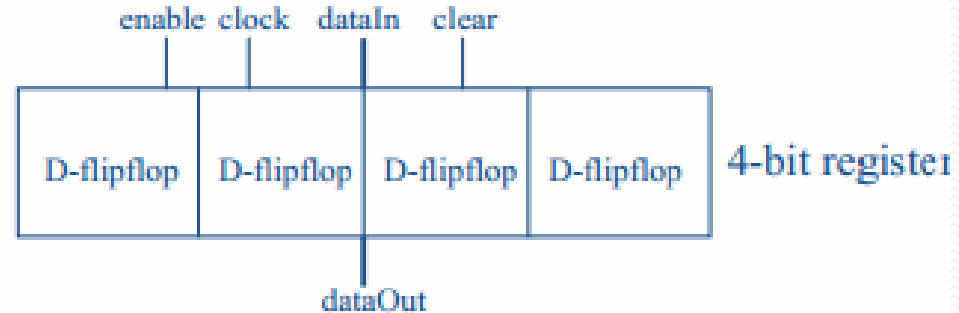
Example: 4-bit ripple counter

■ 4-bit ripple counter

```
module counter(Q , clock, clear);  
output [3:0] Q;  
input clock, clear;  
  
// Instantiate the T flipflops  
t_ff tff0(Q[0], clock, clear);  
t_ff tff1(Q[1], Q[0], clear);  
t_ff tff2(Q[2], Q[1], clear);  
t_ff tff3(Q[3], Q[2], clear);  
  
endmodule
```

Example: Register

■ 4-bit register



```
module register4bits( dataOut, dataIn, enable, clock, clear );  
// Inputs and outputs  
output [3:0] dataOut;  
input [3:0] dataIn;  
input enable, clock, clear;  
// 4 D-flipflops  
edge_dff ff0( dataOut[0], dataIn[0], enable, clock, clear );  
edge_dff ff1( dataOut[1], dataIn[1], enable, clock, clear );  
edge_dff ff2( dataOut[2], dataIn[2], enable, clock, clear );  
edge_dff ff3( dataOut[3], dataIn[3], enable, clock, clear );  
endmodule
```

Example: Register

A 4-bit register using D flip-flops

